CSE 445: Machine Learning

Project Report

SUMMER 2026

Learning a Cost Model for Mobile Package Recommendation in Bangladesh

Decision tree, random forest and polynomial regression compared on 203 live operator packages, deployed as a dependency-free web service.

- **Project Title:** Learning a Cost Model for Mobile Package Recommendation in Bangladesh
- **Project Group / Lead / Submitter:** MD Abdul Wadud (2323372042), submitter; with M. M. Nafis Ridwan (2312584042) and Samiul Haque (2221619642)
- **Course / Section / Department:** CSE 445, Section ___, Department of Electrical and Computer Engineering, North South University
- **Date:** 5 September 2026

github.com/SIUM02/Offer-Analysis

SECTION 2

Abstract

> Bangladesh's four mobile operators publish hundreds of prepaid packages on incompatible terms — different quotas, different validity periods, bundles mixing data, minutes and SMS — so a subscriber cannot tell which costs least for what they need. This project scrapes 287 live packages, defines a cost model that prices any request against any package over a common period, and uses it to label 25,000 synthetic usage profiles. Three models are trained and compared on 5,000 held-out requests: a decision tree reaches 92.9% top-1 agreement, a 150-tree random forest 91.6% top-1 and 97.7% top-3, and a degree-2 polynomial regression 37.8%. Because a wrong package of equal price costs the subscriber nothing, agreement is reported with a monetary measure: the tree's mistakes cost a mean of BDT 9.93, the forest's BDT 25.79. The models are exported to 3.2 MB of JSON and served in pure Python, without scikit-learn, in about a millisecond per request.

Index terms — recommender systems, decision trees, random forest, ridge regression, cost modelling, model deployment.

SECTION 3

Introduction and objectives

Choosing a prepaid package is a comparison problem disguised as a shopping problem. A 7-day pack and a 30-day pack cannot be compared on price; a bundle containing minutes cannot be compared on price-per-gigabyte; and a package that covers everything but SMS may still be the cheapest route once the missing SMS are bought separately. Operators' own pages sort by price, which answers a question nobody asked.

There is no record of what subscribers *should* have bought, so the label is constructed. A cost model computes, for any request and package, the total taka needed to satisfy that request — including repeat purchases when a package is shorter than the period asked for, and the price of buying whatever it leaves out. That model labels every synthetic request with its optimal package, and the machine learning models learn to reproduce the labelling far more cheaply than evaluating it.

OBJ 1 **A verifiable dataset:** every published package from four operators in one schema, no estimated values, every exclusion documented.

OBJ 2 **A cost model defensible in money:** price any request against any package over a common period, so packages of different validity become comparable at all.

OBJ 3 **Three model families compared** on identical features, split and operator masking, reporting agreement *and* monetary error.

OBJ 4 **A working deployment** that answers in milliseconds where scikit-learn cannot be installed.

Contributions. Three things here are not standard coursework practice. A **monetary error metric** alongside accuracy, which reverses the ranking of two models. An **exact dependency-free export**: a fitted forest occupies 395 MB because scikit-learn stores a probability for all 156 classes at every one of 150,592 leaves, yet a leaf

holds only 2.12 classes on average, so the same three models fit in 3.2 MB of JSON executed by standard-library Python. And **honest treatment of a synthetic label** — accuracy here measures agreement with the cost model, not real-world correctness, and the report treats that as a first-class limitation.

SECTION 4

Background

Mobile internet in Bangladesh is sold as prepaid packages rather than monthly contracts, so package selection is a decision repeated every few days over a catalogue that is large, changes often, and is published four incompatible ways — HTML tables, a JSON API, server-rendered tables, and a client-rendered interface with no public API. Comparing by price alone misleads, because the cheapest sticker price is frequently the most expensive way to cover a month.

The underlying problem — constrained selection under a cost function — is classical operations research. What makes it a machine learning problem is deployment economics, since evaluating the cost model means scoring every package for every request while a trained classifier maps a request to a package directly. The models are therefore *function approximators for an explicit optimiser*, which also explains why the trees succeed and the polynomial regression fails: the target is piecewise-constant with hard discontinuities. The method transfers to any domain with a public catalogue, an expressible objective and no record of correct decisions — insurance tiers, cloud instances, utility tariffs.

SECTION 5

Literature review

Theoretical basis. Decision trees induce axis-aligned splits that partition the feature space into regions of constant prediction [1], [2] — the natural hypothesis class for a target defined by thresholds. Random forests average decorrelated trees grown on bootstrap samples with random feature subsets, trading interpretability for variance reduction [3]. Ridge regression fits a least-squares linear model with an L2 penalty [4]; with a degree-2 expansion and one-versus-rest encoding it produces a smooth decision surface, included here precisely because the target is not smooth. All three are used through scikit-learn [5].

Gaps. The recommender-systems literature is dominated by collaborative filtering and content-based methods that learn from observed behaviour [6]; neither applies, as there is no interaction history here. Published work on telecom plan recommendation generally assumes subscriber usage records paired with the held plan — data operators do not release, so it cannot be reproduced from open sources. Where public catalogues are used, comparisons are usually single-dimensional, which this project shows to be misleading: Grameenphone's median data-only package prices at BDT 98/GB against Banglalink's BDT 9, but the gap is largely pack mix, and comparing only packages of 5 GB or more reduces it from roughly 13× to 3×.

Advantage. This work generates labels rather than assuming them and publishes the labelling rule as auditable code. Against shipping the cost model alone, the trained models evaluate faster, generalise to requests never scored, and — by comparing families against one rule — reveal which hypothesis classes can represent it.

SECTION 6

Design methodology

The system has two halves meeting at a single file. Offline, the catalogue is scraped, filtered, and used with the cost model to label 25,000 synthetic requests; three models are fitted and exported. Online, a request arrives from a web form or the command line, the exported model scores every package the chosen operator sells, and the three cheapest of its picks are returned with a costed plan for anything they leave out.

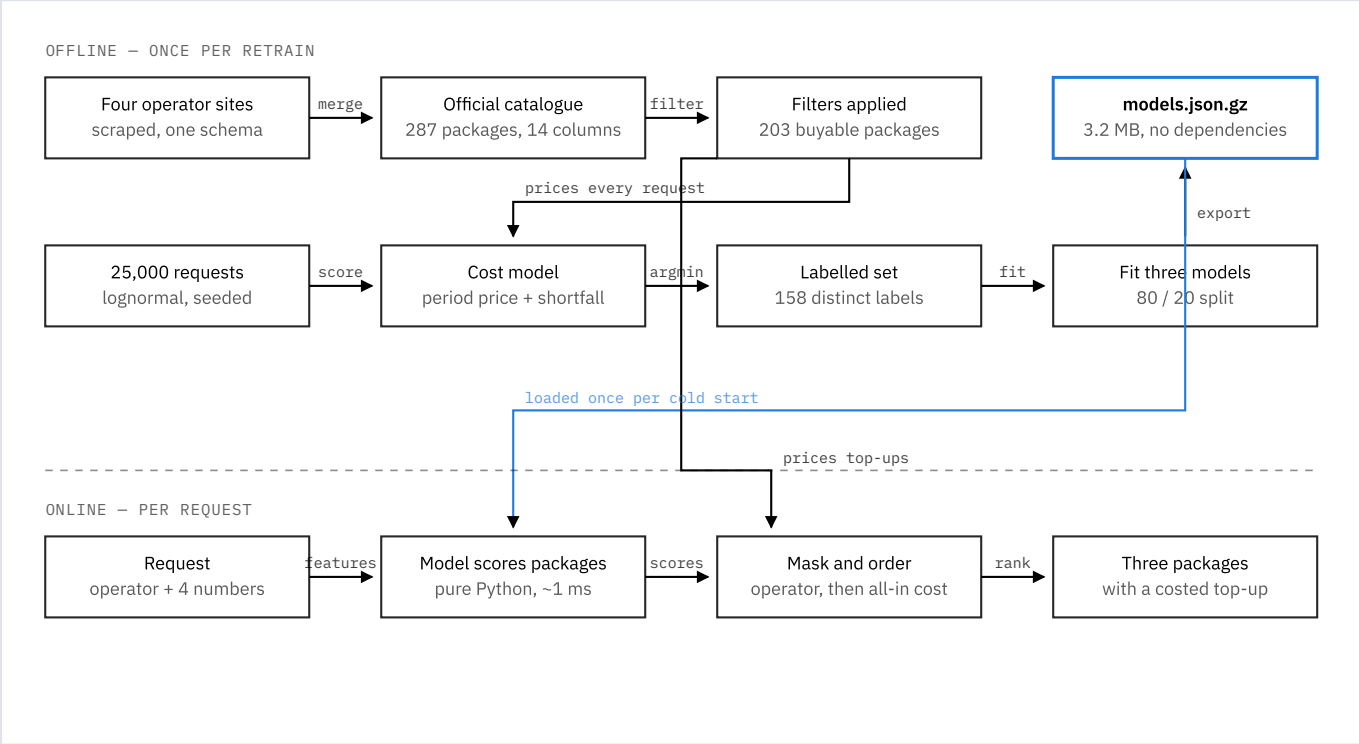

FIG. 1 System architecture. The catalogue feeds both halves: offline it is what the cost model scores to produce labels, and online it is what prices a top-up when the model's package falls short. The two halves communicate only through the exported model file (highlighted), which is what makes the serving side free of machine-learning dependencies.

Preprocessing

Of 287 scraped packages, 84 are excluded. Forty-four belong to categories that sell no quota — `Call Rate` buys a cheaper tariff, `Validity` extends an account, `Bondho SIM` works only on a dormant SIM, and `New SIM`, `Cashback` and `Entertainment` are promotions. Forty more are app-restricted: their quota works only inside particular services, so offering "4 GB: imo + BiP + Telegram only" to somebody who asked for 20 GB of general internet would be wrong even though the number matches. Two normalisations follow: packages marked `Unlimited` store `data_gb = 0` meaning uncapped and are given a large finite quota so they compete on price, and packages with no published validity are treated as monthly rather than dropped.

FIG. 2 Preprocessing. Every exclusion is a rule about what a package *is*, not a statistical filter, and each is reversible by editing one constant in `train_model.py`.

Cost model and labels

Everything is costed over the period requested, which is what makes packages of different validity comparable: a 7-day package covering a 30-day request is bought five times, so it costs five times as much and delivers five times the quota. On top of that period price the model charges for a **shortfall** — priced at the operator's own cheapest package that covers the gap — mild **waste** for unrequested quota, and a mild penalty for being **locked in longer** than asked. Requests are drawn from lognormal distributions, so most are modest with a long tail; a third ask for no minutes and 45% for no SMS, which makes both data-only and bundle packages reachable as labels.

Two tuning decisions were settled by measurement. Meeting the request is a requirement, not a preference: a proportional shortfall penalty fails even at 80×, stalling coverage at 58% of requests a package could have met, while ranking every short package behind every covering one lifted coverage from 22% to the 65% catalogue ceiling. And unit rates must come from single-purpose packages: a bundle of "0.05 GB + 15 minutes" implies a per-gigabyte price in the thousands, and including such rows put Grameenphone's reference rate at BDT 230/GB, making every option score alike.

Features, training and testing

Five features per request — operator (integer-encoded), data in GB, minutes, SMS, validity in days — and the label is the chosen package's `offer_id`; 158 distinct classes appear across the 25,000 requests. The split is 80/20 with a fixed seed, stratified by label, giving 20,000 training and 5,000 test requests. At prediction time every model's scores are masked to the packages the requested operator sells, identically across models, so no model can answer a Robi request with a Grameenphone package.

TABLE I — MODELS COMPARED, ON IDENTICAL FEATURES, SPLIT AND MASKING

MODEL	CONFIGURATION	WHY INCLUDED
Decision tree	`min_samples_leaf = 2`	Axis-aligned splits match a threshold rule; readable
Random forest	`150 trees, min_samples_leaf = 5`	Variance reduction over the single tree
Polynomial regression	`degree 2, standardised, ridge α = 1`	Tests whether a smooth surface can represent the rule

Tools. Python 3.13.7 with scikit-learn 1.9.0, pandas 3.0.5 and NumPy 2.5.1 for training, Matplotlib 3.11.1 for figures. The recommender, the exported models and the web application use the standard library only — `csv`,

`json`, `gzip`, `math`, `http.server` — with no framework. Git and GitHub for version control; Vercel's Python runtime for deployment.

SECTION 7

Testing methodology and results

Success is measured four ways, in the order they matter. **Top-1 and top-3 agreement** on 5,000 held-out requests — the standard measure, and the weakest, because it treats every mistake alike. **Monetary error**: the model's package and the labelled package are both costed for the same request and the difference reported as a mean and 90th percentile, so a correct pick scores zero and an equally-priced substitute is not penalised. **Export fidelity**: `export_models.py` scores 400 random requests through both paths and refuses to write unless the arg-max agrees on every one. **Serving latency** on the deployment target, measured with no machine-learning libraries installed.

Every number in this report is reproduced by two commands, each printing its own results: `python3 scripts/train_model.py` and `python3 scripts/export_models.py`. Seeds are fixed at 42, so a re-run reproduces the tables exactly; training all three takes about eleven seconds.

Results

TABLE II — PERFORMANCE ON 5,000 HELD-OUT REQUESTS, NONE SEEN IN TRAINING

MODEL	TOP-1	TOP-3	MEAN BDT LOST	90TH PCT	FIT TIME
Decision tree	**92.9%**	**95.2%**	**9.93**	**0.00**	**0.11 s**
Random forest (150 trees)	91.6%	97.7%	25.79	0.00	1.05 s
Polynomial regression (degree 2)	37.8%	66.7%	153.86	538.50	0.09 s

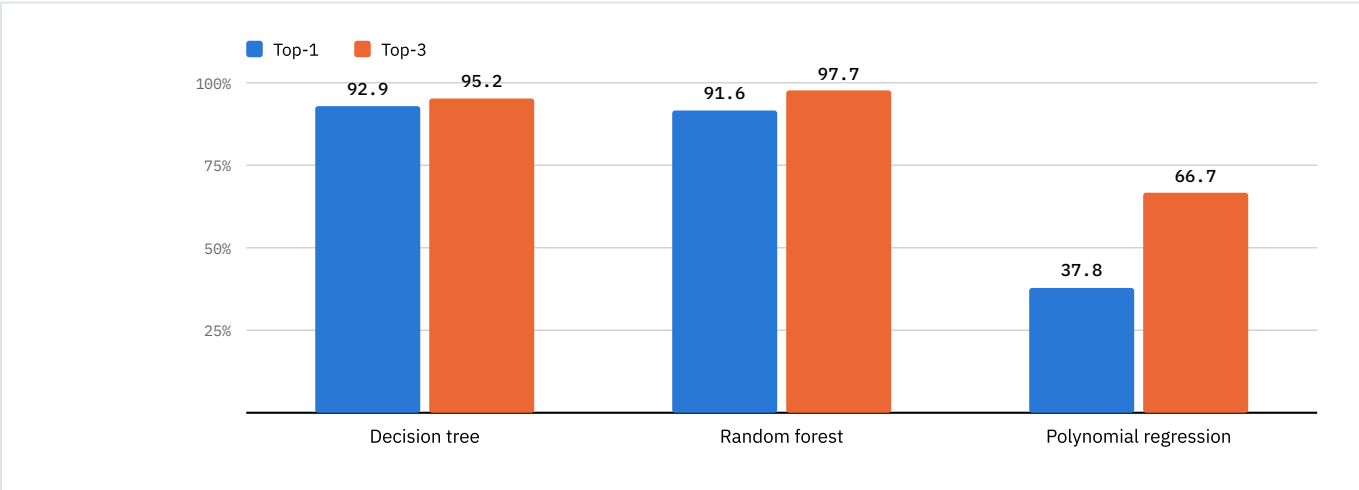

FIG. 3 Agreement with the cost model on held-out requests. The two tree families sit within 1.3 points of each other on top-1; the gap that matters is top-3, where the forest recovers to 97.7%, and the gap that matters more is monetary, in Table II.

The tree families learn the rule almost exactly, as expected: the cost model is piecewise-constant with hard discontinuities — a package is disqualified the moment it falls one SMS short — and axis-aligned splits cut on exactly those boundaries.

The polynomial regression fails, and that is the informative result. A degree-2 surface must separate 158 classes with one smooth function of five variables; it cannot represent a threshold, so it approximates the neighbourhood of the boundary instead. Its monetary error is fifteen times the decision tree's, and its 90th percentile of BDT 538.50 means that in a tenth of cases it costs a subscriber roughly a monthly bundle more than necessary.

Accuracy and cost disagree, and cost is the honest measure. The forest beats the tree on top-3 by 2.5 points yet loses 2.6× more money when wrong: its extra correct picks fall on requests where being wrong was cheap, while its errors land on more expensive packages. Accuracy alone recommends the forest; priced errors recommend the tree.

TABLE III — RANDOM FOREST: ACCURACY AGAINST MODEL SIZE, SAME SPLIT

TREES	MIN LEAF	TOP-1	TOP-3	IN MEMORY	ON DISK
300	2	93.5%	98.8%	1,333 MB	41 MB
150	**5**	**91.6%**	**97.7%**	**395 MB**	**15 MB**
100	8	89.9%	97.0%	191 MB	9 MB
60	12	87.3%	96.4%	86 MB	4 MB

Size is driven by leaf count, not accuracy, so the 300-tree forest needs 1.3 GB of memory to answer one question; the deployed configuration gives up 1.9 points of top-1 for a fifth of the size. Forest feature importances are data 0.272, operator 0.258, minutes 0.185, validity 0.165, SMS 0.119 — operator ranking second is a sanity check, since the label space is partitioned by it.

Deployment. The joblib bundle is 15 MB and unreadable without scikit-learn, which with NumPy, SciPy and pandas exceeds the target platform's 250 MB unzipped limit. Exported as sparse gzipped JSON the same models occupy 3.21 MB with no dependencies: each tree becomes thresholds and child indices, each leaf keeps only the classes that reached it. Across 400 random requests the exported models select the same package 400 times out of 400, with scores differing by at most 2×10^{-16}. On an interpreter with no machine-learning libraries the deployed path answers in 19 ms of cold start, 247 ms on the first request touching the models, and about 1 ms per prediction after.

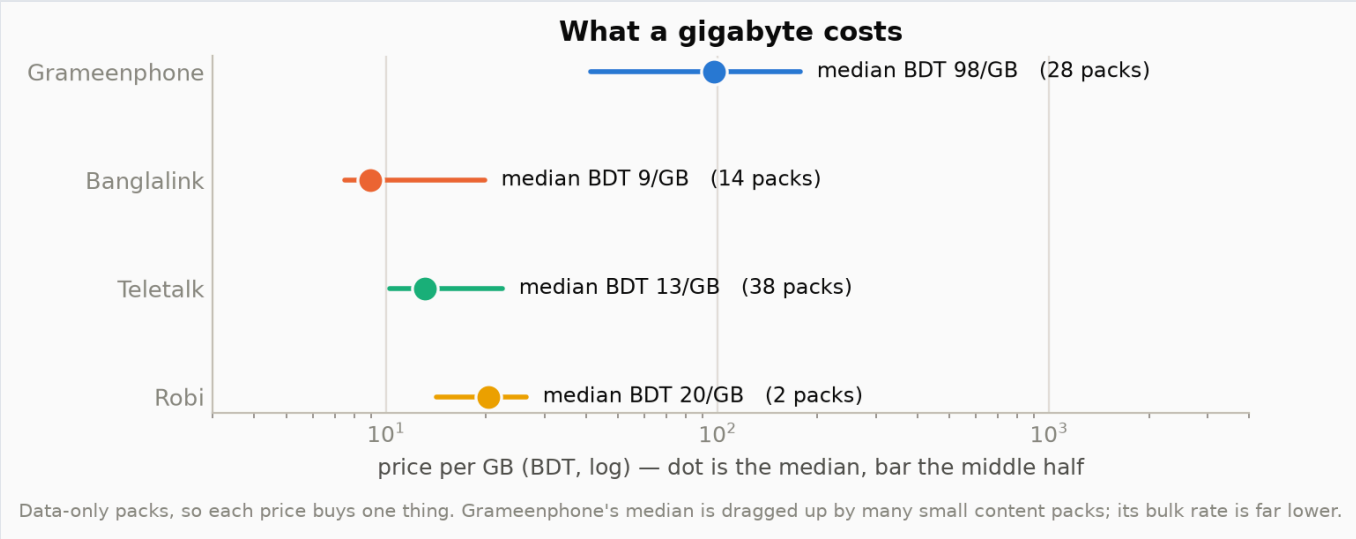

FIG. 4 Price per gigabyte, from packages that sell nothing but data. The headline gap — Grameenphone at BDT 98/GB against Banglalink at BDT 9 — is largely pack mix, and narrows to roughly 3× when only packages of 5 GB or more are compared. This is why the cost model prices a top-up from the 25th percentile of single-purpose packages, not the median.

SECTION 8

Project timeline

TABLE IV — PHASES AND DELIVERABLES

PHASE	DELIVERABLE	DATES
1 · Data collection and merge	`all_operators_official.csv`	24–28 Aug 2026
2 · Cost model and tuning	`score_offers()`	1–2 Sep 2026
3 · Label generation	`training_profiles.csv`	3 Sep 2026
4 · Modelling and comparison	`train_model.py`	4 Sep 2026
5 · Export and deployment	**`models.json.gz, web.py`**	**5 Sep 2026**

> **To complete before submission.** The section number on the title page is still blank.

SECTION 9

Budget

TABLE V — ESTIMATED COST

ITEM	DETAIL	COST (BDT)
Personnel	Student time, unbilled coursework	0
Data	Public operator websites; no dataset purchased	0
Software	Python, scikit-learn, pandas, NumPy, Matplotlib	0
Compute	Training runs in 11 s on a laptop; no cloud GPU	0
Hosting	Vercel free tier; the export is sized to fit it	0
Total	**Zero-cost by design**	**0**

The zero total is an outcome, not an accident: sizing the forest at 150 trees and exporting to plain JSON was decided so the system fits a free hosting tier. A 300-tree forest served through scikit-learn would have needed a paid instance with over 1.3 GB of memory.

SECTION 10

Team members and roles

TABLE VI — ROLES

MEMBER	RESPONSIBLE FOR
MD Abdul Wadud (2323372042)	Data collection, cost model, model training and comparison, export, deployment, report
M. M. Nafis Ridwan (2312584042)	Data collection, model selection, figure plotting
Samiul Haque (2221619642)	Data collection, model selection

SECTION 11

Risks and mitigation

TABLE VII — RISKS, WITH THE MITIGATION IMPLEMENTED

RISK	MITIGATION IN PLACE
Label is synthetic, not observed	Stated in the abstract, results and limitations; the cost model is published as auditable code
Prices go stale	Scrapers are re-runnable; every row records its source page; the catalogue is regenerated, never edited
Model file not portable across library versions	Models exported to plain JSON with no dependency; the exporter verifies fidelity before writing
Deployment exceeds platform limits	Export sized to 3.2 MB; the page states which models loaded and falls back to the cost model, saying so
Uneven operator coverage	Robi's rows documented as a curated public sample; roaming excluded from the table entirely
A recommendation falls short of the request	Every shortfall is reported with the cheapest package that fills it and the all-in total

SECTION 12

Conclusion and limitations

The project delivers a working recommender over a real catalogue: 203 live packages from four operators, a cost model that makes packages of different validity comparable, three models compared on identical terms, and a deployment answering in about a millisecond with no machine-learning library installed. The decision tree is the model to ship — 92.9% top-1 agreement and a mean monetary error of BDT 9.93, fitted in a tenth of a second.

Three limitations bound the claim. **First and most important, accuracy measures agreement with the cost model, not correctness.** The catalogue records what operators sell, never what a subscriber should have bought, so every percentage here reads as "reproduces the cost model this often"; if the cost model is wrong about what subscribers value, a model reproducing it perfectly is wrong the same way. **Second, the catalogue is a snapshot and one operator's is partial** — Robi's public listing is personalised, so conclusions of the form "Robi offers less" are unsupported. **Third, requests are generated, not collected**; the lognormal usage distribution is an assumption, and real data would change the label distribution and probably the model ranking.

The next step follows from the first limitation: replace synthetic requests with real request-and-purchase records, at which point the same pipeline, unchanged, trains a genuinely empirical model. Publishable weight would come from that substitution plus a user study comparing the recommender's picks with what subscribers actually choose; as it stands the contribution is a reproducible method and an honestly-scoped result, not a state-of-the-art claim.

SECTION 13

References

[1] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[2] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] A. E. Hoerl and R. W. Kennard, “Ridge regression: biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[5] F. Pedregosa *et al.*, “Scikit-learn: machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] F. Ricci, L. Rokach and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York: Springer, 2015.

[7] Grameenphone Ltd., grameenphone.com; Banglalink Digital Communications Ltd., banglalink.net; Teletalk Bangladesh Ltd., teletalk.com.bd; Robi Axiata Ltd., robi.com.bd. [Online]. Accessed: 3 Sep. 2026.

[8] Project source code and data, “Offer-Analysis,” GitHub repository, 2026. [Online]. Available: https://github.com/SIUM02/Offer-Analysis

---

Source, data and figures: github.com/SIUM02/Offer-Analysis. Figures are produced by `scripts/plot_dataset.py` and every number by `scripts/train_model.py` or `scripts/export_models.py`. Seeds are fixed; a re-run reproduces the tables exactly.